

Introduction to Node.js and Express

Server-Side JavaScript

Chapter Overview

- 01** Node.js Runtime and Modules
- 02** Project Setup End-to-End
- 03** Introduction to Express
- 04** Request Lifecycle and Middleware
- 05** Async Programming and Error Handling
- 06** Security and Production Readiness



Node.js Runtime and Modules

Section 1



What is Node.js?

JavaScript Outside the Browser

- Node.js runs JavaScript on the server using Chrome's V8 engine
- Event-driven, non-blocking I/O — designed for scalable network applications
- Event loop handles I/O concurrency; `worker_threads` available for CPU-bound tasks

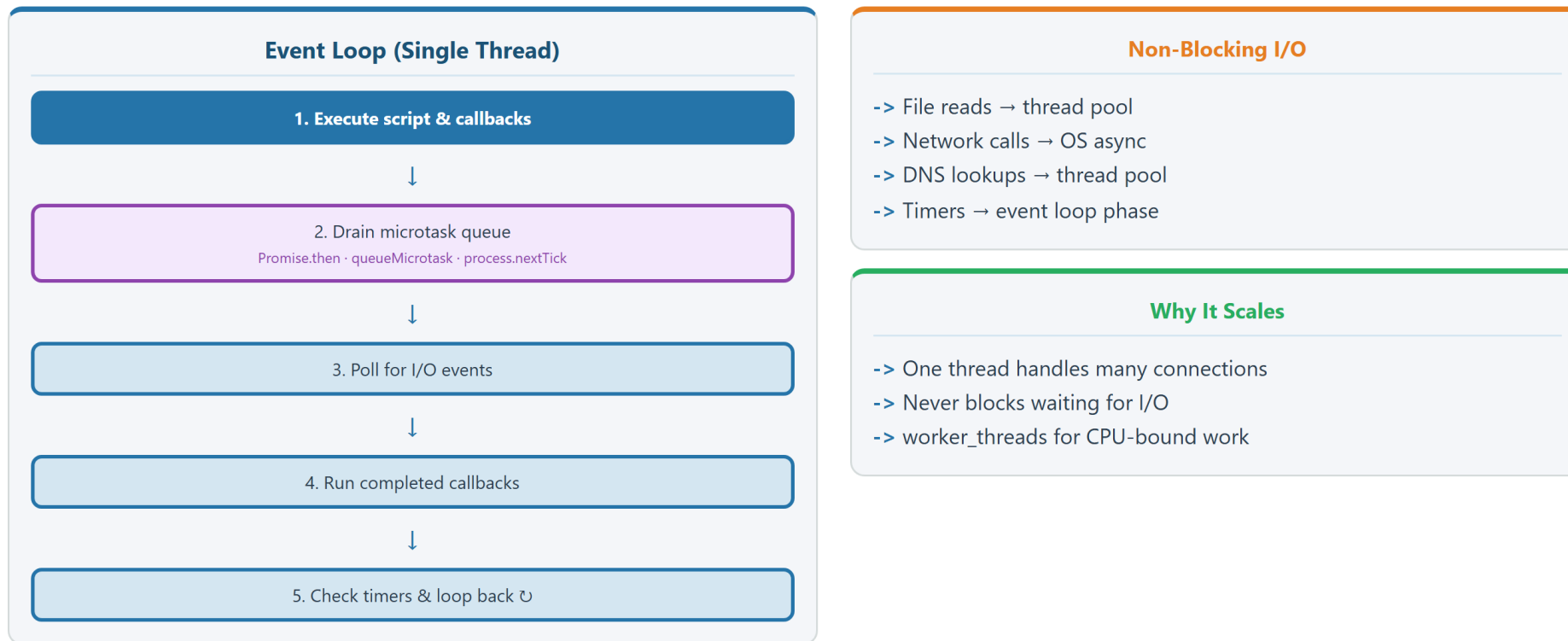
Why Node.js?

- Same language on frontend and backend — JavaScript everywhere
- Massive ecosystem — npm has millions of packages
- Excellent for APIs, microservices, and real-time applications



Node.js Event Loop

Node.js Event Loop



libuv provides the thread pool (default 4 threads) · Microtasks drain between every phase



Your First Node.js Script

```
// hello.js (ESM – requires "type": "module" in package.json)
import os from 'node:os';
import path from 'node:path';

console.log('Hello from Node.js!');
console.log(`Platform: ${os.platform()}`);
console.log(`Node version: ${process.version}`);
console.log(`CPUs: ${os.cpus().length}`);
console.log(`Current dir: ${path.resolve('.')}`);

// Run: node hello.js
```



Node.js Module System

ES Modules (import / export) — The Modern Default

- Set "type": "module" in package.json to enable ESM for all .js files
- import express from 'express' — static analysis, tree-shaking friendly
- Use node: protocol for built-ins: import fs from 'node:fs/promises'

CommonJS (require / module.exports) — Legacy

- Still used by many existing npm packages and older tutorials
- require() is synchronous and dynamic — can appear inside if blocks

Built-in Modules

- fs, path, http, os, crypto, node:test — no npm install needed



Module Systems Comparison

Module Systems: CommonJS vs ES Modules

CommonJS (CJS)

Default in Node.js

```
// Export
const add = (a, b) => a + b;
module.exports = { add };

// Import
const { add } = require('./math');
```

- > Synchronous loading
- > .js extension (default)
- > Dynamic — can require() inside if blocks

ES Modules (ESM)

Modern Standard

```
// Export
export const add = (a, b) => a + b;

// Import
import { add } from './math.js';
```

- > Async-capable loading
- > .mjs or "type": "module"
- > Static — analyzed at build time

ESM is the standard for new projects — set "type": "module" in package.json

Creating and Using Modules

```
// math.js – named exports
export function add(a, b) { return a + b; }
export function multiply(a, b) { return a * b; }

// app.js – import and use
import { add, multiply } from './math.js';
console.log(add(3, 5));      // 8
console.log(multiply(4, 6)); // 24
```



Project Setup End-to-End

Section 2



npm — Node Package Manager

Initializing a Project

- `npm init -y` creates a `package.json` file with defaults
- `package.json` stores metadata, scripts, and dependency versions

Installing Packages

- `npm install express` — adds Express as a production dependency
- `npm install -D nodemon` — adds nodemon as a dev-only dependency
- `npm ci` — clean install from lock file (use in CI pipelines for reproducible builds)

Key Files

- `package-lock.json` — exact dependency tree (always commit this file)
- `node_modules/` — installed packages (add to `.gitignore`, never commit)



Project Setup from Scratch

```
# 1. Create project directory and initialize
mkdir my-api && cd my-api
npm init -y

# 2. Enable ES Modules – add to package.json:
#   "type": "module"

# 3. Install Express and dev tools
npm install express
npm install -D nodemon

# 4. Add start scripts to package.json
#   "scripts": {
#     "start": "node src/index.js",
#     "dev": "nodemon src/index.js"
#   }

# 5. Create source directory
mkdir src && touch src/index.js
```



Minimal Express Server

```
// src/index.js
import express from 'express';

const app = express();
const PORT = process.env.PORT || 3000;

// Parse JSON request bodies (required for POST/PUT)
app.use(express.json({ limit: '1mb' }));

app.get('/', (req, res) => {
  res.json({ message: 'API is running', status: 'ok' });
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```



Express Project Structure

Express Project Structure

Project Structure

```
my-api/  
├─ src/  
│   ├─ index.js      # Entry point  
│   └─ routes/  
│       ├─ users.js   # User routes  
│       └─ health.js  # Health check  
└─ middleware/  
    └─ logger.js      # Custom middleware  
├─ package.json       # Dependencies  
├─ package-lock.json  # Lock file  
├─ .env               # Secrets (not committed)  
├─ .gitignore         # Ignore node_modules/  
└─ node_modules/      # Installed pkgs
```

package.json Scripts

```
"scripts": {  
  "start": "node src/index.js",  
  "dev": "nodemon src/index.js",  
  "test": "jest"  
}
```

.gitignore Essentials

```
node_modules/  
.env  
dist/  
*.log
```



Introduction to Express

Section 3



What is Express?

Minimal Web Framework

- Express is a thin layer over Node's http module — adds routing, middleware, and convenience methods
- Unopinionated — you choose how to structure your app

Core Concepts

- Routes — map HTTP methods and URL patterns to handler functions
- Middleware — functions that process requests before they reach route handlers
- Request/Response — enhanced objects with helper methods like `res.json()`



Routing in Express

HTTP Methods

- `app.get()` — retrieve data (safe, idempotent)
- `app.post()` — create new resources
- `app.put()` — replace an existing resource entirely
- `app.delete()` — remove a resource

Route Parameters and Query Strings

- `app.get('/users/:id')` — `:id` becomes `req.params.id`
- `/search?q=node&limit=10` — accessed via `req.query.q` and `req.query.limit`



Route Matching and Ordering

Routes Match Top-Down

- Express checks routes in the order they are defined — first match wins
- Put specific routes BEFORE parameterized routes (/users/me before /users/:id)

Handling 404 — Not Found

- Add a catch-all middleware AFTER all routes to handle unmatched paths
- `app.use((req, res) => res.status(404).json({ error: 'Not found' })))`

Router Modules

- `express.Router()` groups related routes into separate files for large apps



Express Routing Map

Express Routing Map — RESTful API

GET POST PUT DELETE

Method	Route	Handler	Description
GET	/api/users	List all	Returns array of users
GET	/api/users/:id	Get one	req.params.id → find user
POST	/api/users	Create	req.body → new user (201)
PUT	/api/users/:id	Update	Replace user fields
DELETE	/api/users/:id	Remove	204 No Content

Params: req.params.id Query: req.query.limit Body: req.body.name ⚠ Routes match top-down — first match wins



Complete CRUD API

```
import express from 'express';
import { randomUUID } from 'node:crypto';

const app = express();
app.use(express.json({ limit: '1mb' }));

let users = [
  { id: randomUUID(), name: 'Alice', email: 'alice@example.com' },
];

app.get('/api/users', (req, res) => {
  res.json(users);
});

app.get('/api/users/:id', (req, res) => {
  const user = users.find(u => u.id === req.params.id);
  if (!user) return res.status(404).json({ error: 'Not found' });
  res.json(user);
});
```



CRUD API — Create, Update, Delete

```
app.post('/api/users', (req, res) => {
  const { name, email } = req.body; // express.json() required
  if (!name || !email) {
    return res.status(400).json({ error: 'name and email required' });
  }
  const user = { id: randomUUID(), name, email };
  users.push(user);
  res.status(201).json(user);
});

app.put('/api/users/:id', (req, res) => {
  const idx = users.findIndex(u => u.id === req.params.id);
  if (idx === -1) return res.status(404).json({ error: 'Not found' });
  users[idx] = { ...users[idx], ...req.body };
  res.json(users[idx]);
});

app.delete('/api/users/:id', (req, res) => {
  users = users.filter(u => u.id !== req.params.id);
  res.status(204).end();
});
```

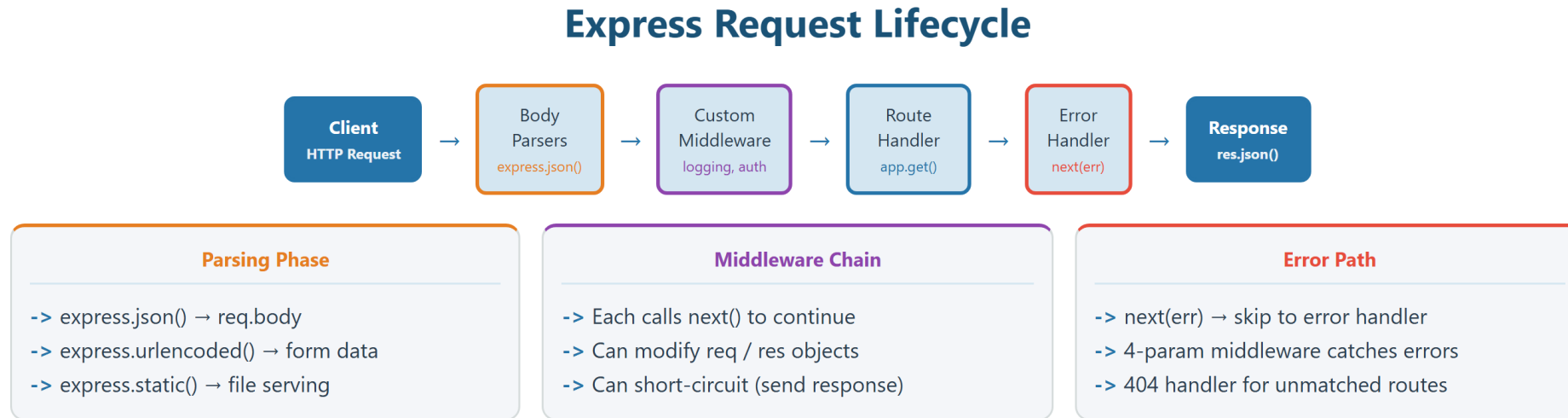


Request Lifecycle and Middleware

Section 4



Express Request Lifecycle



Middleware order matters: helmet → cors → parsers → auth → routes → 404 → error handler

Understanding Middleware

What is Middleware?

- Functions with signature (req, res, next) that run in a pipeline
- Each middleware can read/modify the request, send a response, or call next()
- Rule: every middleware must either call next() or send a response — never neither

Ordering Matters

- Middleware runs in the order registered — place body parsers before routes
- If a middleware doesn't call next(), the request stops there

Scope

- app.use() — applies to ALL routes
- app.use('/api', middleware) — applies only to routes under /api



Built-in and Third-Party Middleware

Express Built-ins

- `express.json()` — parses JSON bodies (must be registered before routes using `req.body`)
- `express.urlencoded({ extended: true })` — parses HTML form submissions
- `express.static('public')` — serves static files (HTML, CSS, images)

Popular Third-Party Packages

- `cors` — enables Cross-Origin Resource Sharing for frontend apps
- `morgan` — HTTP request logging for debugging and monitoring
- `helmet` — sets security-related HTTP headers



Custom Middleware Example

```
// Logging middleware – runs on every request
function requestLogger(req, res, next) {
  const start = Date.now();
  res.on('finish', () => {
    const ms = Date.now() - start;
    console.log(`${req.method} ${req.url} ${res.statusCode} - ${ms}ms`);
  });
  next(); // Pass control to the next middleware
}

// Register middleware BEFORE routes
app.use(express.json({ limit: '1mb' }));
app.use(requestLogger);

// Now define routes...
app.get('/api/health', (req, res) => {
  res.json({ status: 'ok' });
});
```



Parsing Request Bodies

JSON Input (API Clients, fetch)

- `app.use(express.json())` — parses Content-Type: application/json
- Without this, `req.body` is undefined — a common beginner mistake

Form Input (HTML Forms)

- `app.use(express.urlencoded({ extended: true }))` — parses form data
- Handles Content-Type: application/x-www-form-urlencoded

Input Validation

- Always validate `req.body` before using it — check required fields exist



Handling JSON and Form Data

```
import express from 'express';
import { randomUUID } from 'node:crypto';

const app = express();

// Register BOTH parsers before routes
app.use(express.json({ limit: '1mb' }));
app.use(express.urlencoded({ extended: true }));

// JSON body from fetch() or axios
app.post('/api/users', (req, res) => {
  const { name, email } = req.body;
  if (!name || !email) {
    return res.status(400).json({ error: 'Missing fields' });
  }
  res.status(201).json({ id: randomUUID(), name, email });
});
```

Async Programming and Error Handling

Section 5



Why Async Matters in Node.js

Single-Threaded Event Loop

- Node.js runs one thread — blocking it means zero requests are processed
- I/O operations (file reads, DB queries, HTTP calls) must be non-blocking

The Callback Problem

- Nested callbacks create deeply indented code that's hard to read (callback hell)
- Error handling is inconsistent across nested callbacks

Modern Solutions

- Promises and `async/await` provide clean, readable asynchronous code



Async/Await

```
// Fetching with await
const res = await fetch('https://api.example.com/users');
// Always check the response status before parsing
if (!res.ok) {
  throw new Error(`HTTP ${res.status}: ${res.statusText}`);
}
const data = await res.json();

// Full async function with error handling
async function getUsers() {
  try {
    const res = await fetch('https://api.example.com/users');
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return await res.json();
  } catch (err) {
    console.error('Failed to fetch users:', err.message);
    return [];
  }
}
```



Async File Operations

fs/promises API

- `import { readFile } from 'node:fs/promises'` — Promise-based file system API
- `await readFile('data.json', 'utf-8')` — reads without blocking the event loop

Common Async Operations

- File I/O, database queries, HTTP requests, timer delays
- `Promise.all()` — run multiple independent operations in parallel

Note on `fetch()`

- `fetch()` is a built-in global in all current (Node.js v18+) Node.js versions — no external package needed



Async Express Route Handlers

```
import { readFile } from 'node:fs/promises';

// Express 5 catches async errors automatically – no wrapper needed
app.get('/api/config', async (req, res) => {
  const raw = await readFile('config.json', 'utf-8');
  res.json(JSON.parse(raw));
});

// Parallel async operations
app.get('/api/dashboard', async (req, res) => {
  const [users, orders] = await Promise.all([
    db.find('users'), db.find('orders')
  ]);
  res.json({ users, orders });
});

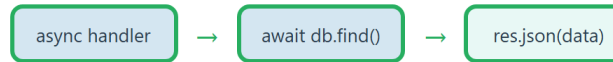
// If any await rejects, Express 5 calls next(err) for you
// The error reaches your centralized error handler automatically
```



Async Error Propagation

Async Error Propagation in Express

✓ Happy Path



✗ Error Path (Express 5 — automatic)



Route Handler (Express 5)

```
// No try/catch needed
app.get('/api/items', async (req, res) => {
  const items = await db.find();
  res.json(items);
});
// Rejections auto-forwarded
```

Error Handler

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.status || 500)
    .json({ error: err.message });
});
```

⚠ Must have 4 parameters

Express Error Handling

Error-Handling Middleware

- Four-parameter signature: (err, req, res, next) — signals an error handler
- Must be registered AFTER all routes with `app.use()`

Forwarding Errors

- In synchronous code: throw an error or call `next(err)`
- In async code: Express 5 catches rejected promises automatically — no wrapper needed

Best Practice — RFC 9457 Problem Details

- Return structured errors: { type, title, status, detail } for consistent API responses
- Never send stack traces to clients in production — log them server-side



Centralized Error Handler

```
// 404 handler – AFTER all routes
app.use((req, res) => {
  res.status(404).json({
    type: 'https://api.example.com/errors/not-found',
    title: 'Not Found',
    status: 404,
    detail: `No route matches ${req.method} ${req.path}`,
  });
});

// Centralized error handler – MUST have 4 params (Express convention)
app.use((err, req, res, next) => {
  console.error('Unhandled error:', err.stack);
  const status = err.status || 500;
  res.status(status).json({
    type: 'https://api.example.com/errors/internal',
    title: status === 500 ? 'Internal Server Error' : err.title,
    status,
    detail: process.env.NODE_ENV === 'production'
      ? 'An unexpected error occurred' : err.message,
  });
});
```



Process-Level Error Safety Nets

Unhandled Promise Rejections

- `process.on('unhandledRejection')` catches missed `await/catch` errors
- Log the error and shut down gracefully — the process is in an unknown state

Uncaught Exceptions

- `process.on('uncaughtException')` fires on unhandled synchronous throws
- Do NOT use this to keep running — always exit after logging
- Use a process manager like PM2 or Docker to auto-restart on crash

Production Recommendation

- Combine with health checks and monitoring — never silently swallow errors



Graceful Shutdown Pattern

```
const server = app.listen(PORT, () => {
  console.log(`Listening on port ${PORT}`);
});

// Log and exit – do NOT attempt to recover
process.on('unhandledRejection', (reason) => {
  console.error('Unhandled Rejection:', reason);
  server.close(() => process.exit(1));
});

process.on('uncaughtException', (err) => {
  console.error('Uncaught Exception:', err);
  server.close(() => process.exit(1)); // Always exit
});

// Graceful shutdown on SIGTERM/SIGINT (Docker, Kubernetes, Ctrl+C)
for (const sig of ['SIGTERM', 'SIGINT']) {
  process.on(sig, () => {
    console.log(`${sig} received, shutting down...`);
    server.close(() => process.exit(0));
  });
}
```



Security and Production Readiness

Section 6



Security Essentials

HTTP Security Headers

- Use helmet middleware — sets Content-Security-Policy, X-Frame-Options, and more
- Enable CORS only for trusted origins — never use cors() with no options in production

Rate Limiting

- express-rate-limit — prevents brute-force and DDoS attacks per IP
- Apply globally or per-route (e.g., stricter limits on /auth endpoints)

Dependency Security

- npm audit — checks installed packages for known vulnerabilities
- npm audit fix — automatically installs patched versions where possible



Security Middleware Setup

```
import express from 'express';
import helmet from 'helmet';
import cors from 'cors';
import rateLimit from 'express-rate-limit';

const app = express();

// Security headers
app.use(helmet());

// CORS – restrict to known origins
app.use(cors({ origin: process.env.ALLOWED_ORIGIN }));

// Rate limiting – 100 requests per 15 min per IP
app.use(rateLimit({ windowMs: 15 * 60 * 1000, max: 100 }));

// Body parsing with size limit
app.use(express.json({ limit: '1mb' }));
```

Input Validation with Zod

Why Validate?

- `req.body` is untrusted user input — never pass it directly to your database
- Validation prevents injection attacks, type confusion, and data corruption

Zod — Runtime Schema Validation

- Define schemas that describe exactly what shape your data should have
- `schema.parse(data)` throws on invalid input — use in middleware or routes
- TypeScript-first but works great with plain JavaScript too



Zod Validation Example

```
import { z } from 'zod';
import { randomUUID } from 'node:crypto';

const UserSchema = z.object({
  name: z.string().min(1).max(100),
  email: z.string().email(),
  age: z.number().int().min(0).optional(),
});

// Validation middleware
function validate(schema) {
  return (req, res, next) => {
    const result = schema.safeParse(req.body);
    if (!result.success) {
      return res.status(400).json({ errors: result.error.issues });
    }
    req.body = result.data; // Use parsed & typed data
    next();
  };
}

app.post('/api/users', validate(UserSchema), (req, res) => {
  // req.body is guaranteed valid here
  res.status(201).json({ id: randomUUID(), ...req.body });
});
```



Environment Configuration

Environment Variables

- Store secrets in environment variables — never hardcode API keys or passwords
- Use `node --env-file .env` in development, platform env vars in production
- Set `NODE_ENV=production` to disable verbose error messages

Configuration Best Practices

- Load and validate all config at startup — fail fast on missing values
- Never commit `.env` files — add to `.gitignore`, share a `.env.example` template



Where Express Fits (2026)

Express in the Ecosystem

- Express 5 — current stable release (GA since late 2024); native async error handling, improved path matching
- Express 4 — previous LTS, still widely deployed; massive middleware ecosystem

Alternatives Worth Knowing

- Fastify — 2-3× throughput, built-in schema validation, strong TypeScript support
- Hono — ultrafast, runs on Edge/Workers/Bun/Node, modern web standard APIs

Recommendation

- Express is the right choice for learning — patterns transfer to any framework
- Evaluate alternatives when building new production services



Key Takeaways

Key Takeaways

1. Node.js runs JavaScript on the server with a non-blocking event loop
2. Use ESM (import/export) with "type": "module" — the modern Node.js standard
3. npm init + npm install sets up a project — always commit package-lock.json
4. `express.json({ limit: '1mb' })` must be registered before any route reading `req.body`
5. Middleware runs in order: helmet → cors → parsers → routes → error handler
6. Express 5 catches async errors natively — rejected promises reach your error handler automatically
7. Validate all input (Zod), set security headers (helmet), and rate-limit your API

